

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/378226734>

QoS and Fairness Oriented Dynamic Computation Offloading in the Internet of Vehicles Based on Estimate Time of Arrival

Article in IEEE Transactions on Vehicular Technology · July 2024

DOI: 10.1109/TVT.2024.3364669

CITATIONS

17

READS

68

6 authors, including:



Weizhe Zhang

Harbin Institute of Technology

241 PUBLICATIONS 4,430 CITATIONS

SEE PROFILE



Hui He

Harbin Institute of Technology

110 PUBLICATIONS 1,397 CITATIONS

SEE PROFILE



Rahul Yadav

Harbin Engineering University

37 PUBLICATIONS 1,252 CITATIONS

SEE PROFILE

QoS and Fairness Oriented Dynamic Computation Offloading in the Internet of Vehicles based on Estimate Time of Arrival

Chen Ling, Weizhe Zhang, Hui He, Rahul Yadav, Jiayin Wang, Desheng Wang

Abstract—The rapid development of Internet of Vehicles (IoV) and Mobile Edge Computing (MEC) enables vehicles to offload their applications to Roadside Units (RSU). However, without knowledge of vehicles' future locations, wireless connections between vehicles and RSUs are too vulnerable to finish offloading processes. Most existing methods allocate computation tasks statically, ignoring latency between task reception and processing in real IoV environments. Moreover, multiple services running on RSUs limit the availability of resources for offloading which require an efficient allocation of limited available resources. Therefore, this paper addresses the vehicular computation offloading problem by predicting vehicle mobility. We first introduce the Estimated Time of Arrival service into the vehicular MEC architecture to predict vehicles' driving state. Second, we formulate vehicular computation offloading as a dynamic process that efficiently allocates the limited available resources and selects the offloading location for each computation task to balance QoS and response fairness. Thirdly, the cooperation among RSUs in our work optimizes the task QoS, but it increases the search space causing a high time consumption. We use the Dynamic Programming algorithm to optimize QoS and response fairness in the cooperative computation offloading problem and propose two offloading algorithms. To obtain the performance of our algorithms close to real IoV environments, we propose a fine-grained simulation method based on Veins. We evaluate our and existing algorithms based on real datasets from Erlangen, Germany, and Shenzhen, China.

Index Terms—Edge Computing, Internet of Vehicular, Computation Offloading, QoS Optimization, Vehicular Networks

I. INTRODUCTION

WITH the development of Internet of Vehicles (IoV), vehicles can operate various applications, such as navigation, environment-aware, Argument Reality (AR) [1]. The applications could access information and computing resource among vehicles through the wireless modules equipped by vehicles. Unfortunately, due to the limited computing and storage capabilities, vehicles cannot process these applications simultaneously while maintaining the Quality of Services (QoS). However, vehicular Mobile Edge Computing (MEC) is introduced to overcome vehicle capacity limitations [2].

C. Ling, and H. He are with the School of Computer Science and Technology, Harbin Institute of Technology, Xidazhi Street, Harbin, China.

D. Wang is with School of Computer Science and Technology, Harbin Institute of Technology, Shenzhen, Guangdong, China.

W. Zhang is with the School of Computer Science and Technology, Harbin Institute of Technology, Xidazhi Street, Harbin, China and the Department of New Networks, Peng Cheng Laboratory, Shenzhen, Guangdong, China.

R. Yadav is with the College of Computer Science and Technology, Harbin Engineering University, Harbin, China

J. Wang is with the School of Cyberspace Science, Harbin Institute of Technology, Xidazhi Street, Harbin, China.

Vehicular MEC emerges as a paradigm to provide computing and storage resources to the vehicles, which can alleviate the vehicles' workload and optimize the applications' QoS simultaneously through the Roadside Units (RSU) close to vehicles. The RSUs possess more computational and storage capabilities than vehicles [3] [4] [5]. In vehicular MEC architectures [2], the applications generate the computation tasks and the vehicles offload these tasks to RSUs for computation to maximize applications' throughput and minimize service latency.

Because of directly relating to driving behaviors, navigation applications play an important role in vehicle center display and operation. The navigation applications require high-precision maps to accurately position the vehicles, which has a large data size. In detail, navigation applications analyze high-precision maps and provide high-precision routes for vehicles, especially unmanned vehicles. With high-precision routes, vehicles can safely and efficiently travel to their destinations, and the detailed road information of high-precision routes can avoid accumulative errors in driving behaviors. Considering the limited storage space of vehicles, offloading the navigation applications to the RSUs is an effective method, optimizing their QoS, as shown in Fig. 1. Otherwise, the vehicles can only store the maps with coarse-grained details due to their limited storage space and decrease the analysis accuracy of high-precision routes, increasing the driving risk. Without RSUs, navigation applications even can't provide enough accuracy for unmanned vehicles [6] [7]. An ideal process refers to offloading the tasks generated by navigation applications to the nearest RSUs and sending back the results to the vehicles before leaving the RSUs' coverage. Moreover, part of the resources on RSUs is already occupied by other services, like access control of high-precision maps. The offloading process should allocate the limited available resources to the tasks generated by navigation applications. However, the vehicles' mobility impacts the process of sending back the results, decreasing the applications' QoS. In conclusion, to predict vehicle mobility and to optimize workload distribution, an efficient solution for vehicular computation offloading is desperately required.

A. Motivations and Problems

Some researchers have proposed vehicular computation offloading methods to solve the vehicle mobility problem [8] [9] and optimize the applications' performance [10] [11].

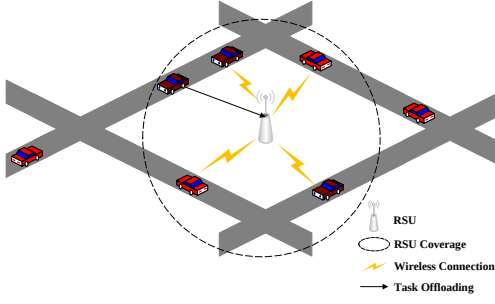


Fig. 1: The Scene of Vehicular Computation Offloading

Firstly, the applications generate the computation tasks and offload them to the RSUs. Then, these methods formulate the vehicular computation offloading problem model based on vehicle mobility, tasks' requirements, and the limited available resources of RSUs. The vehicular computation offloading methods solve the problem model and generate the tasks' offloading locations. However, current vehicular computation offloading methods are not efficient in real-world circumstances.

The drawbacks of the existing vehicular computation offloading methods: First of all, most methods only consider the driving state within several seconds. In detail, these methods offload the tasks to RSUs, and the task results are transmitted to vehicles before leaving the RSUs' coverage. However, for computation-intensive tasks with long processing time, these methods cannot predict the vehicles' locations when tasks are finished. At that time, the vehicles may already leave the RSUs' coverage, causing the failure of task results reception. It may further influence the usability of applications. Second, most researchers [12] [13] formalize the vehicular computation offloading problem as a static task allocation problem based on the limited available resources of RSUs. The existing vehicular computation offloading methods have fixed decision windows. In each decision window, the tasks are received from the vehicles, and the tasks are offloaded to RSUs to maximize the tasks' performance at the end of the decision window. However, the latency between task reception and offloading decision is ignored, increasing the tasks' offloading latency. In real vehicular MEC architectures, vehicles usually require the RSUs to process the tasks as soon as they are received by RSUs. Third, most vehicular computation offloading methods are simulated using Python or MATLAB with real or randomly generated datasets [12]. Nevertheless, this kind of simulation method is coarse-grained, ignoring the network status changes caused by vehicle mobility. **The network delay and QoS of computation tasks are calculated for computation tasks in each decision window. Moreover, the simulated offloading algorithms can only access vehicles' current positions, resulting in inaccurate simulation results.** The vehicular computation offloading methods should be verified in an environment close to real-world circumstances.

These drawbacks motivate us to propose a new vehicular computation offloading method. However, there are three main challenges. First, the computation-intensive tasks require

more processing time, requiring the vehicular computation offloading method to predict the vehicles' location when computation-intensive tasks are finished. The current prediction method cannot satisfy the requirements of computation-intensive tasks. Therefore, the first challenge of our work is **CH1**: *how to predict vehicle mobility to satisfy the requirements of computation-intensive tasks*. Second, vehicular computation offloading is a dynamic process. It requires the vehicular computation offloading method to allocate the limited available resources to tasks and generate the offloading decision for each task as soon as it is received to decrease the latency between task reception and processing. Moreover, the vehicular offloading method should also optimize the tasks' performance to guarantee the QoS of navigation applications. So that the second challenge of our work is **CH2**: *how to dynamically allocate the limited available resources of RSU and offload the tasks with low latency to optimize the tasks' performance*. Finally, the current simulation method of vehicular computation offloading is coarse-grained and far from the real world. In a fine-grained simulation method of vehicular computation offloading, the vehicle mobility and network status should be simulated simultaneously with the real vehicle trajectory. Therefore, the third challenge is **CH3**: *how to build a simulated IoV environment with real vehicle trajectory and simulate the vehicular computation offloading with network status changes*.

B. Contributions

This paper proposes a vehicular MEC architecture to optimize the QoS of navigation applications and the response fairness of RSU. In this vehicular MEC architecture, we design two offloading protocols to organize the message delivery of the offloading process. The key contributions of our work are summarized as follows:

- Targeting **CH1**, we introduce the Estimated Time of Arrival (ETA) [14] service into our vehicular MEC architecture and propose an ETA-based vehicular computation offloading model. The ETA service could predict the vehicle arrival time for each intermediate node of the vehicle route, whose accuracy is higher than 85% [15] [16]. By introducing the ETA service, the offloading algorithm could accurately sense the vehicle's location when computation-intensive tasks are finished, promising the usability of navigation applications.
- Targeting **CH2**, we propose two dynamic vehicular computation offloading algorithms, named Dynamic programming maximum QoS and Fairness for Cooperative offloading (DQFC) and Greedy maximum QoS and Fairness for Independent offloading (GQFI). When one task is generated, our algorithms dynamically select the RSUs to process this task, minimizing the latency between task reception and processing. Meanwhile, considering the part of resources on RSUs already occupied by other services, the proposed algorithms allocate the limited available resources of RSUs to optimize both the applications' QoS and the RSUs' response fairness. Furthermore, DQFC considers the cooperation between RSUs and offloads

each computation task to several RSUs, increasing the RSUs' limited available resource utilization and performing better performance than GQFI.

- Targeting **CH3**, we propose a vehicular computation offloading simulation method based on Veins [17] with the vehicle trajectories in Shenzhen, China [18]. The proposed method provides the fine-grained simulation of network status and vehicle mobility for each computation task. **Compared with the existing coarse-grained simulation method, our simulation method could simulate the message transmission of each computation task, generating accurate simulation results. Moreover, the simulated offloading algorithms could access the positions and speeds of vehicles in real-time, resulting in an accurate calculation of transmission rate and network delay.** Through the proposed vehicular computation offloading simulation method, we could obtain the performance of offloading algorithms close to real-world circumstances.

The experiments conducted on real-world datasets show that the proposed algorithms, GQFI and DQFC, increase the QoS and fairness factor by 15.02% and 52.82% relatively compared with an existing method based on Evolution Genetic Algorithm [3]. Moreover, the tasks' completion ratio is averagely increased by 87.17%, guaranteeing the usability of applications.

Furthermore, this paper is organized as follows: Section 2 describes related work on vehicular computing offloading. Section 3 describes the design of our vehicular MEC architecture. In Section 4, we introduce the problem model of independent vehicular computation offloading. The algorithm procedures of GQFI are described in section 5. Section 6 describes the problem model of cooperative vehicular computation offloading and the algorithm procedure of DQFC. Section 7 introduces the design of the dynamic vehicular computation offloading simulation method. The performance of GQFI and DQFC with existing algorithms is compared in section 8. Finally, section 9 summarizes our work and gives the future direction.

II. RELATED WORK

This section summarizes the research on vehicular computing offloading. There are two categories: vehicular edge offloading and volunteer offloading based on IoV [19]. The research on vehicular edge computing offloading selects the RSUs as offloading positions based on the task property and RSU states. In contrast, the research on volunteer computing offloading based on IoV selects the other vehicles as the offloading position and considers vehicle mobility in offloading processes.

A. Vehicular Edge Offloading

Vehicular edge computing offloading has been exploited by recent work to optimize task QoS. Wu et. al. [20] proposed a mixed-offloading paradigm in a wireless-powered mobile edge computing network to maximize residual energy gains by allocating the offloading position and transmission power. However, device mobility is ignored in this paradigm, influencing the tasks' QoS. Lakhan et. al. [13] proposed a mobility-aware blockchain-enabled offloading method to optimize the

communication and computation cost of applications and ensure privacy. However, the proposed method only focuses on the synchronization offloading process, decreasing the offloading performance in a highly dynamic environment. Huang et. al. [21] proposed an offloading scheme among the edge servers based on multi-agent reinforcement learning to select the global optimal offloading location and optimize the edge servers' energy consumption. However, device mobility is ignored in this scheme. Nan et. al. [22] formulate the vehicular task offloading problem as non-convex mixed integer nonlinear programming and propose two offloading algorithms to generate the offloading decisions. However, they assume that the vehicles follow an exponential distribution, which has a long distance from the real world. Qu Yuan Luo et. al. [23] focused on the offloading from vehicles to RSUs and proposed a vehicular computation offloading method based on particle swarm optimization to minimize the latency and costs. Nevertheless, the proposed method ignored the vehicle mobility. Yang et. al. [24] proposed a degradation-of-QoS (DeQoS) attack against VANETs, which can establish invalid connections between attackers and RSUs and waste the resources of RSUs. Moreover, they designed a cross-layer relay-resistant authentication protocol based on distance-bounding technique to defense against this DeQoS attack.

Wu et. al. [25] propose a content caching method to place the content cache to RSU and optimize the network latency by considering the file characteristics and network conditions in heterogeneous vehicular networks. Nevertheless, they ignore the content cache updating problem in their method. Materwala et. al. [3] designed an offloading algorithm based on an Evolutionary Genetic Algorithm that optimizes energy consumption and maintains applications' Service Level Agreement. However, it may not be verified in real-world environments. Misra et. al. [9] proposed a mobility-aware task offloading method to optimize offloading latency in the vehicular network. However, the proposed scheme didn't consider cooperation among the fog nodes. Yang et. al. [26] proposed an edge-assisted decentralized authentication (EADA) architecture to provide secure and more communication-efficient authentication services. Moreover, the proposed architecture considers both delay minimization and single point of failure in the vehicular authentication process.

B. Volunteer Offloading based on IoV

In volunteer computing offloading based on IoV methods, the researchers utilize the multi-hop transmission in offloading between vehicles to further optimize resource utilization. Song et. al. [27] designed the edge server placement and offloading in edge networks based on Genetic Algorithm, using a transmission with a lower cost than the wired transmission. However, the time consumption of the offloading decision is ignored in their method. Li et. al. [12] designed a vehicular edge computing offloading paradigm to maximize the benefit of computing offloading between vehicles and RSUs. However, they ignored the computing tasks' QoS, resulting in low performance of computing offloading. Chen et. al. [28] proposed an end-edge-cloud architecture of vehicles

and an Asynchronous Advantage ActorCritic Algorithm based offloading algorithm on optimizing the offloading latency and response fairness. However, they ignore the influence of vehicle mobility on processing delay. However, the sending-back process is ignored by this scheme. Chen et. al. [29] focused on the computing offloading among the vehicles and proposed an offloading scheme using vehicle-to-vehicle communication to optimize the processing time by selecting the optimal offloading ratio and location. Nevertheless, they did not consider future vehicle mobility in their task offloading scheme. Wu et. al. [30] proposed several offloading algorithms based on reinforcement learning to optimize the workload distribution of vehicles statically.

However, the task results' sending back process is ignored in their algorithm, which may influence the offloading performance. Luo et. al. [31] proposed a self-learning-based computing offloading scheme between vehicles and RSUs to generate each vehicle's best response decision and minimize the joint offloading cost. Nevertheless, the mobility of vehicles should be considered in their scheme. Xu et. al. [32] proposed a wireless resource allocation method to decrease the channel interference among vehicles and RSUs based on reinforcement learning. Nevertheless, they didn't consider the future positions of the vehicles. Zhang et. al. [33] introduced fiber-wireless technology in vehicular computing offloading and a software defined network based task offloading scheme to reduce the offloading latency. However, the proposed scheme requires that the vehicles' mobility follows a synchronized random mobility model, which is challenging to be satisfied in real-world environments. Zhan et. al. [8] proposed a mobility-aware offloading method to obtain the optimal offloading decision and balance offloading latency and energy consumption. However, they ignore the sending back process of the task results in their method.

In conclusion, these works have some critical shortages:

- Vehicle mobility isn't deeply considered in current vehicular computation offloading methods. Most existing methods only predict the vehicle's location within several seconds, causing the sending back failure of computation-intensive tasks' results.
- The vehicular computation offloading is formulated as a static task allocation problem far from the real world. The research on the dynamic vehicular computation offloading problem has limited progress.
- Most vehicular computation offloading methods weren't verified in fine-grained simulation environments. Current vehicular computation offloading simulation methods cannot provide network status changes caused by vehicle mobility.

III. THE VEHICULAR MEC ARCHITECTURE

This section introduces our vehicular MEC architecture. We first introduce the network topology of vehicular MEC environments in detail. Then we explain the proposed procedure of the computation offloading scheme. **The key notations are listed in Table I.**

TABLE I: Notations in this paper

Notation	Meaning
V	Vehicle set
R	RSU set
r_k	An RSU
f_k	available CPU computing capability of r_k
S_k	available Storage capacity of r_k
Q_k	Task queue of r_k
v_i	A vehicle
$route_i$	ETA result of v_i
$n_i^{\tau_m}$	Position of v_i in time τ_m
t_j	A computation task
cpu_j	Computing resource required by t_j
mem_j	Input data size of t_j
d_j	Deadline position requirement of t_j
T_{jk}	The transmission rate between $n_i^{\tau_j}$ and r_k
P_t	The transmission power of RSUs and vehicles
P_n	The noise power of RSUs and vehicles
$D(n_i^{\tau_m}, r_k)$	The distance between point τ_m to RSU r_k
UD_{jk}	The upload delay of task t_j to proxy RSU r_k
RD_j	The relay delay of task t_j
A_{jk}^r	The set of relay RSUs and target RSU of task t_j to RSU r_k
A_j	Available RSU list of task t_j
MD_{jk}	The communication delay of task t_j
PD_{jk}	The processing delay of task t_j on target RSU r_k
QD_{jk}	The queue delay of task t_j on target RSU r_k
CD_{jk}	The computation delay of task t_j on target RSU r_k
TD_{jk}	The total delay of task t_j on target RSU r_k
QoS_j	The QoS of task t_j on target RSU r_k in independent vehicular computation offloading
$normQoS_j$	The normalized QoS of task t_j on target RSU r_k in independent vehicular computation offloading
QoS'_j	The QoS of task t_j on target RSU r_k in cooperative vehicular computation offloading
$normQoS'_j$	The normalized QoS of task t_j on target RSU r_k in cooperative vehicular computation offloading
F	The Fairness factor of all RSUs
$minF_j$	The minimum fairness factor acquired by task t_j
$normF_j$	The normalized fairness factor of task t_j on target RSU r_k
p_j	The total performance of task t_j on target RSU r_k
α	The weight coefficient of total performance
X_j	The decision vector of task t_j
x_{jk}	The decision variable of task t_j on RSU r_k in independent vehicular computation offloading
x'_{jk}	The decision variable of task t_j on RSU r_k in cooperative vehicular computation offloading
C_m	The coverage of RSUs

A. System Architecture Design

The proposed vehicular MEC architecture focuses on navigation applications, generating computation-intensive tasks with long processing times. Considering the hardware condition limitation and complex scenario of IoV, offloading the tasks of navigation applications to trusted RSU devices could acquire higher performance. Moreover, navigation applications can be modeled as video streaming applications, whose QoS is directly related to the transmission rate [34]. Considering the features of navigation applications mentioned above, the following assumptions are made:

- Each vehicle has a specific destination and route, consisting of several intermediate nodes. We define the

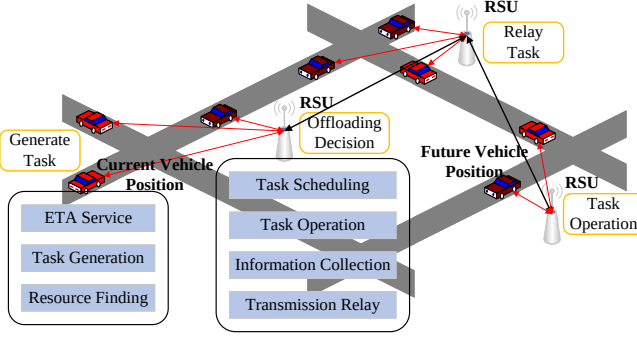


Fig. 2: The Vehicular MEC Architecture

intermediate nodes as the intersection between the road segments within vehicle routes.

- The tasks generated by the navigation applications operating on the vehicles require high storage space, whose tasks can only be processed on RSUs.
- The RSU must finish the tasks before the vehicle enters the tasks' deadline positions. Considering that the service process is continuous, the transmission rate will directly affect the QoS.
- When the RSUs receive multiple tasks, the task that arrives first is processed first.

Based on these assumptions, we propose a vehicular MEC architecture shown in Fig. 2.

Fig. 2 shows a section of roads with several RSUs deployed at specified intervals. The vehicles consist of three modules, ETA service, task generation, and resource-finding. The ETA service module accesses the vehicles' routes and predicts the arrival time of intermediate nodes. In detail, The ETA service module utilizes the traffic information of each road segment to estimate the arrival time based on recurrent neural network. Moreover, the traffic information and parameters are periodically collected by RSUs from vehicles and sensors equipped on RSUs, consisting of average vehicle speed, weather conditions, and road closures [35]. Then, the vehicles obtain the traffic information from the RSUs and store the traffic information in the storage space occupied by ETA modules. Comparing with high-precision map, the traffic information and parameters have smaller data size, which can be easily stored on each vehicle. In addition, the task-generation module receives the tasks generated by the navigation applications. Finally, the resource-finding module searches the proxy RSUs and transmits the task to the closest RSU.

Moreover, the RSUs consist of four modules, task-scheduling, task-operation, information-collection, and task-relay. The information-collection module accesses the state of other RSUs, parses the task information, and inputs them into the task-scheduling module. Then, the task-scheduling module uses the offloading algorithm to select the target RSU to process the task. After that, the task is transmitted to the target RSU through several RSUs' task-relay modules. Finally, the task-operation module processes the task and sends the task results back to the vehicle.

B. Independent Vehicular Computation Offloading Protocol Design

Based on the above system architecture, we design the independent vehicular computation offloading protocol to organize the message delivery of the offloading process. The independent vehicular computation offloading protocol includes three phases: information exchanging, offloading decision, and task processing.

- 1) **Information Exchanging:** In vehicular MEC architecture, the information-collection module in RSUs periodically broadcasts the position and available resource information to vehicles within their coverage through the control channels [36]. Because of using the control channels, the continuous information exchanging phase won't create high network overheads. When the vehicle receives the broadcasting message, its resource-finding module records it into the RSU state list. Then the vehicle will send back its position information to RSUs, and RSUs will also record its position. Moreover, the RSUs also periodically exchange their operation state with each other through wired connections, just like Kubernetes. In detail, each RSU periodically broadcasts its operation state to other nearby RSUs through wired connections. Then, the nearby RSUs act as intermediate RSUs to forward the operation state to RSUs that are far from these nearby RSUs. In addition, the RSUs also update the state list after receiving operation states. Because operation states have small data size, the information exchanging between RSUs also have low network overheads.
- 2) **Offloading Decision:** Once navigation applications generate the tasks through the task-generation module, the vehicles formalize the task messages and filter the RSUs based on the ETA service. With the ETA result of each intermediate node, the vehicles can filter out the RSUs that cannot send back task results before vehicles leave RSUs' coverage. Then, vehicles send the task messages and available RSU list to the closest RSU, called proxy RSU. After receiving the task message, the proxy RSU searches for the optimal target RSU in available RSUs to process the task based on the decision algorithm in the task-scheduling module. Moreover, the proxy RSU may not be able to send the task message to the target RSU directly. Therefore, task transmission needs several relay RSUs to forward the task message.
- 3) **Task Processing:** Through the task-relay module of relay RSUs, the target RSU will finally receive the task message. Then, the task-operation module in target RSU inserts this task into the task queue and processes it when RSU's CPU is idle. After the task is finished, the target RSU will send the results back to the vehicle when arriving in the target RSU's coverage.

The entire flow chart of the proposed independent offloading protocol is illustrated in Fig. 3.

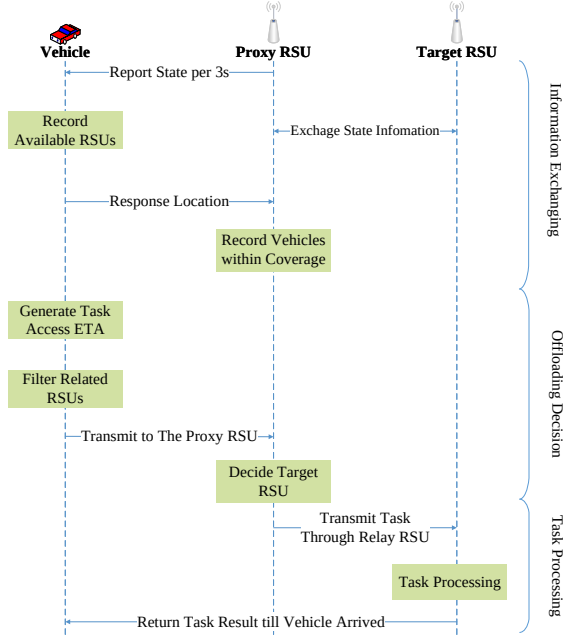


Fig. 3: The Independent Vehicular Computation Offloading Protocol

IV. PROBLEM MODEL OF INDEPENDENT VEHICULAR COMPUTATION OFFLOADING

We denote the set of vehicles as V , and the set of RSUs is R . $|V|$ and $|R|$ are the total number of vehicles and RSUs. The RSU $r_k \in R$ is characterized by a tuple $r_k = (f_k, S_k, Q_k)$. In detail, f_k is the limited available computing resource possessed by r_k , S_k is the limited available storage capacity of r_k , and Q_k is the task queue of r_k . Moreover, vehicles usually have certain travel routes to their destination in urban areas provided by electric map applications, such as Google map and Baidu map [37]. Based on the ETA service of electric map applications, the vehicle $v_i \in V$ can estimate the arrival time of each intermediate node, which is $route_i$, and $n_i^{\tau_m} \in route_i$ is an intermediate node of v_i with an estimate arrival time τ_m . In addition, each vehicle generates the task randomly, characterized by a tuple $t_j = (cpu_j, mem_j, d_j)$. cpu_j represents the computing resource required by t_j , mem_j is the input data size of t_j . In addition, $d_j = n_i^{\tau_j}$ is the deadline position requirement, requiring task t_j to be finished before the vehicle enters the deadline position.

A. Communication Model

This paper assumes each task possesses a fixed portion of bandwidth resources [38]. Therefore, the transmission rate of vehicle v_i to RSU r_k at intermediate node $n_i^{\tau_m}$ is

$$T_{mk} = \min \left(W \cdot \log_2 \left(1 + \frac{P_t}{D(n_i^{\tau_m}, r_k)^h P_n} \right), T_D \right), \quad (1)$$

where W is the bandwidth of the wireless connections between RSUs and vehicles. P_t and P_n is the transmission and noise power, respectively. Moreover, to simplify the transmission

rate calculation, we use the distance between vehicles and RSUs to calculate the path loss. $D(n_i^{\tau_m}, r_k)$ is the distance between intermediate node $n_i^{\tau_m}$ and RSU r_k , and T_D is the theoretic maximum transmission rate. h indicates the path-loss exponent. As shown in equation 1, the closer between vehicles and RSUs, the higher the transmission rate will be obtained by vehicles and RSUs. However, in vehicular network transmission, the transmission rate is also limited by the channel capacity, coding scheme, and transmission time slot, which has a theoretical maximum value [39]. Then, we investigate the existing research on vehicular computing offloading and define the communication and computation delays [8]- [11]. In detail, the delays correlate with the input data size of tasks, transmission rate, and limited available computing resource possessed by RSUs. Moreover, the RSUs must receive and process all input data to generate the task results. Therefore, task t_j 's upload delay is

$$UD_{jk} = \frac{mem_j}{T_{mk}}. \quad (2)$$

After the task t_j 's transmission, RSU r_k will select the target RSU to process this task. There are two possibilities. First, the target RSU is proxy RSU, meaning the communication delay of t_j is its upload delay. Second, the target RSU is another RSU, so the proxy RSU must transmit t_j to the target RSU through several relay RSUs. The transmission between RSUs is through a wired network, meaning a maximum transmission rate. So that the relay delay of RSUs is

$$RD_{jk} = \frac{mem_j}{T_D} \cdot |A_{jk}^r|, \quad (3)$$

where A_{jk}^r is the set of relay RSUs, including all the relay RSUs and the target RSU needed by task t_j . Moreover, the transmission window size limits the maximum transmission rate of RSUs [39]. The data sent or received by RSUs are temporarily stored in this transmission window to wait for further processing. Considering that the RSUs are mainly designed for wireless transmission to vehicles, the transmission window size of RSUs is also limited, which can only meet the requirement of wireless transmission. Therefore, the maximum transmission rate of wired connections is also decided by the maximum transmission rate of the wireless transmission rate. We assume that the maximum transmission rate of wired connections equals that of the wireless transmission. Although the relay transmission increase the offloading delays of the tasks, it enables the proxy RSU searches the target RSU with higher QoS. Moreover, the computation delay on proxy RSU may exceeds the vehicle's resident time, causing the failure of sending back process. The relay transmission could search the RSUs with computation delay below the vehicle's resident time, increasing the completion ratio of computation offloading. Therefore, the communication delay of t_j can be formalized as

$$MD_{jk} = UD_{jk} + RD_{jk}. \quad (4)$$

B. Computation Model

After the task transmission, RSUs store the tasks' data in memory and process these data. Then, the target RSU queues and processes all the received tasks in order. First, we define the processing delay of task t_j as

$$PD_{jk} = \frac{cpu_j}{f_k}. \quad (5)$$

Then, we define the task's time consumption in the queue waiting. We set $Q_k = \{t_1, t_2, \dots, t_j\}$. Based on this, the queue delay of t_j is the processing delay summation of the tasks before t_j , shown as

$$QD_{jk} = \sum_{l=1}^{j-1} PD_{lk}. \quad (6)$$

In addition, the computation delay of task t_j is the summation of queue delay and processing delay, shown as

$$CD_{jk} = QD_{jk} + PD_{jk}. \quad (7)$$

When the target RSU finishes the task t_j , the result of task t_j will be sent back to the vehicle, and its used storage space will be released. Finally, the total delay of task t_j is

$$TD_{jk} = MD_{jk} + CD_{jk}. \quad (8)$$

C. Problem Formalization

In detail, considering that the navigation applications are continuous, the task results sending back process is also continuous. Therefore, we assume that the target RSUs keep sending back the task results until the vehicle leaves the current road. Furthermore, the task QoS depends on the current road length and transmission rate. The task QoS is shown as

$$QoS_j = \sum_{k=1}^{|R|} \frac{1}{L} \int_{L_{begin}}^{L_{end}} x_{jk} \cdot T_{lk} \, dl. \quad (9)$$

As shown above, L is the length of the current road. L_{begin} and L_{end} are the starting and ending points of the current road. In addition, x_{jk} represents the offloading decision between task t_j and RSU r_k . If $x_{jk} = 1$, task t_j will select RSU r_k as the target RSU. Otherwise, the task t_j will not be processed by RSU r_k . In our vehicular MEC architecture, only one x_{jk} equals to 1, and others are 0. The task QoS is the transmission rate of sending back per of unit road length. The closer the current road is to the target RSU, the higher the QoS of the task. In addition, to further optimize the performance of vehicular MEC architecture, we also define the response fairness factor as the variance of RSUs' queue length, shown as

$$F_j = \frac{K}{\sum_{k=1}^{|R|} (|Q_k| + x_{jk} \cdot PD_{jk} - |\bar{Q}|)^2}, \quad (10)$$

$|\bar{Q}|$ is the average processing delay summation of all RSUs. $|Q_k|$ is the processing delay summation of RSU r_k . In our

vehicular MEC architecture, the task should simultaneously consider QoS and response fairness. Therefore, these two metrics should be normalized to formulate a total performance of the task. We perform min-max normalization to the QoS and response fairness [40]. The normalized QoS is shown below.

$$normQoS_j = \frac{QoS_j}{T_D}. \quad (11)$$

As shown above, because the transmission rate of tasks cannot exceed the maximum transmission T_D , the value of normalized QoS is $[0, 1]$. Then, we calculate the minimum fairness factor acquired by the task to normalize the fairness factor. In detail, for each available RSU, we assume that the task is offloaded to it, append the task to its task queue, and calculate the fairness factor of this assumption. The detailed minimum fairness factor is shown below.

$$minF_j = \min_{r_k \in A_j} \left(\frac{K}{\sum_{r_l \in R, k \neq l} (|Q_k| - |\bar{Q}|)^2 + (CD_{jk} - |\bar{Q}|)^2} \right), \quad (12)$$

where A_j is the available RSU list of the task t_j . The maximum fairness factor is infinity, meaning all RSUs' queue delays are same. Moreover, to normalize the fairness factor to $[0, 1]$, we take reciprocal of fairness factor in min-max normalization. Then, the normalized fairness factor is

$$normF_j = 1 - \frac{minF_{jk}}{F_{jk}}. \quad (13)$$

Based on this, the range of the normalized fairness factor is $[0, 1]$. Then we define the task's total performance.

$$p_j = \alpha \cdot normQoS_j + (1 - \alpha) \cdot normF_j. \quad (14)$$

In this equation, $\alpha \in [0, 1]$ is a coefficient parameter representing QoS's importance, which is initialized from the environmental parameters. If $\alpha = 1$, offloading process will only optimize QoS and ignore the response fairness. In addition, if $\alpha = 0$, offloading process will only optimize response fairness and ignore the other.

Our vehicular MEC architecture aims to optimize the performance of the offloading process and the whole architecture simultaneously. Therefore, the following optimization problem is formalized.

$$\max_{x_j} p_j \quad (15)$$

$$\text{s.t. } x_{jk} mem_j \leq S_k \quad (15\text{-c1})$$

$$D(n_i^{\tau_\eta}, r'_k) \leq C_m, \quad x_{jk} D(n_i^{\tau_\theta}, r_k) \leq C_m \quad (15\text{-c2})$$

$$n_i^{\tau_\eta} \leq n_i^{\tau_\theta} \leq d_j \quad (15\text{-c3})$$

$$x_{jk} TD_{jk} \leq \tau_\theta - \tau_\eta \quad (15\text{-c4})$$

$$r_k \in A_j \quad (15\text{-c5})$$

$$x_{jk} \in \{0, 1\}, \quad \sum x_{jk} = 1. \quad (15\text{-c6})$$

This equation represents the offloading problem model of task t_j . Task t_j is generated at $n_i^{\tau_\eta}$, proxied by RSU r'_k , and processed on RSU r_k . Moreover, RSU r_k sends back the result

of task t_j when vehicle v_i arrives at $n_i^{\tau_\theta}$, we call this node as send-back location. In addition, X_j is the decision vector of task t_j , whose elements are x_{jk} . Moreover, the integral operators in p_j should consider the road where vehicle located. We set all RSUs to send the task results back to vehicles when they are located on the nearest roads to RSUs among their routes. Based on this, we translate the integral operators to the optimization equation in 15.

Moreover, constraint 15-c1 constraints that the data size of task t_j cannot exceed the limited available storage space of RSU r_k . The constraint 15-c2 constraints that the distance between vehicle v_i and RSUs r_k, r'_k cannot exceed the RSUs' coverage C_m . Then, the constraint 15-c3 constraints that the result of task t_j must be sent back when vehicle v_i is between generating point $n_i^{\tau_\eta}$ and deadline position d_j . In addition, constraint 15-c4 requires the task to be finished before the vehicle arrives at send-back location. Furthermore, in constraint 15-c5, the target RSU is in the available RSU list. Finally, the constraints 15-c6 introduces the value of decision vector X_j .

V. INDEPENDENT OFFLOADING DECISION ALGORITHM

After receiving the task message and available RSU list, the proxy RSU will run the decision algorithm to get the target RSU with the optimal total performance. As shown in equation 15, the solution space of the decision algorithm is the available RSU list. Hence, traversing the available RSU list can find the target RSU with maximum total performance in a polynomial processing time. The offloading algorithm needs to greedy search all available RSUs and return the target RSU with the highest performance. We propose a greedy complete offloading decision algorithm to search the target RSU with the optimal performance, named Greedy maximum QoS and Fairness for Independent offloading (GQFI) algorithm. The detailed procedure of GQFI is shown in Algorithm 1.

Algorithm 1: GQFI algorithm

input : $route_i$, the ETA results of v_i ; r'_k , proxy RSU of t_j ; A_j , set of available RSUs; $t_j = (cpu_j, mem_j, d_j)$, the task property; α , weight coefficient.

output: $Target$: the optimal target RSU; L : send-back location.

```

1  $Target \leftarrow NULL, accessState(A_j)$ ;
2 for  $r_k \in A_j$  do
3   if  $S_k > mem$  then
4      $PD_{jk} \leftarrow cpu_j / f_k', QD_{jk} \leftarrow \sum_{t_l \in Q_k} \frac{cpu_l}{f_{kl}}, UD_j \leftarrow \frac{mem_j}{T_{jk}}, RD_j \leftarrow relayPath(r'_k, r_k) \cdot \frac{mem_j}{T_D}$ ;
5      $TD_{jk} \leftarrow PD_{jk} + QD_{jk} + UD_j + RD_j, minDist \leftarrow C_m$ ;
6     for  $n_i^{\tau_m} \in route_i$  do
7       if  $D(n_i^{\tau_m}, r_k) < C_m$  and  $total\_delay_{jk} < \tau_m$  and  $D(n_i^{\tau_m}, r_k) < minDist$  then
8          $minDist \leftarrow D(n_i^{\tau_m}, r_k), L \leftarrow n_i^{\tau_m}.road$ ;
9          $normQoS_j \leftarrow \frac{QoS_j}{T_D}, normF_j \leftarrow 1 - \frac{minF_j}{F_j}$ ;
10         $p_j \leftarrow \alpha \cdot normQoS_j + (1 - \alpha) normF_j$ ;
11        if  $p_j > Target.performance$  then
12           $Target \leftarrow r_k, Target.performance \leftarrow p_j$ ;
13 return  $Target, L$ 

```

As shown in Algorithm 1, GQFI first accesses all available RSUs' working states (line 1). Then, it calculates the PD_{jk} , QD_{jk} , UD_{jk} , and RD_{jk} of each available RSU in line 4. After that, the search for the relay path is finished based on the longitude first strategy. With the relay path, GQFI calculates

the TD_{jk} of each RSU (line 5). After calculating TD_{jk} , GQFI needs to find the send-back location of each available RSU. In detail, vehicle v_i should arrive at the send-back location later than TD_{jk} and be within the RSUs' coverage C_m . Moreover, the send-back location also should have the minimum distance with available RSUs to have a better QoS. Therefore, the intermediate node that satisfies the delay requirement and has a minimum distance with available RSUs is assigned as the send-back location (lines 6-8). Afterward, GQFI calculates the normalized QoS and fairness factor of each RSU based on service location, compares it with the current optimal target RSU, and updates the optimal target RSU (lines 10-13). After QoS and fairness factor calculation of all available RSUs, GQFI returns the optimal target RSU $Target$ and its send-back location L .

In addition, GQFI traverses the available RSU list first. Then, GQFI traverses the intermediate node of ETA results within each available RSU coverage. Therefore, the time complexity of GQFI also depends on the length of the available RSU list and ETA results. Assuming the length of the available RSU list and ETA results are K and T , the time complexity of GQFI is $O(TK)$.

VI. COOPERATIVE VEHICULAR COMPUTATION OFFLOADING

In the independent offloading decision scheme, the task can only be processed on one RSU, causing a high processing delay. The high processing delay causes a late send-back location, even violating the deadline position. Therefore, to further optimize the offloading decision scheme, we propose a cooperative offloading protocol to decrease the processing delay, enabling one task to be processed by multiple RSUs in parallel. In the cooperative offloading protocol, one task can be processed by multiple RSUs in parallel, and each RSU processes part of the task. In detail, based on the existing works [41] [29], each task can be divided into several sub-tasks with different offloading ratios. Then, the sub-tasks are transmitted to several RSUs responsible for the processing.

A. Cooperative Vehicular Computation Offloading Protocol Design

Comparing with the independent offloading protocol, the cooperative offloading protocol mainly improves the offloading decision procedure. In detail, we add the task message division operation to the proxy RSUs to split the task message into several sub-task messages corresponding to the target RSUs. Then, the proxy RSU transmits each sub-task message to its target RSU independently. In addition, compared with the independent offloading protocol, dividing tasks into several sub-task decrease the offloading delay and increases the response fairness. However, some sub-tasks may require more relay RSUs than independent offloading protocol, increasing the relay delay. The cooperative offloading protocol selects the target RSUs whose total delays are lower than the vehicles' resident time to decrease the influence of relay delay increment. **The detailed cooperative offloading protocol is shown in Fig. 4.**

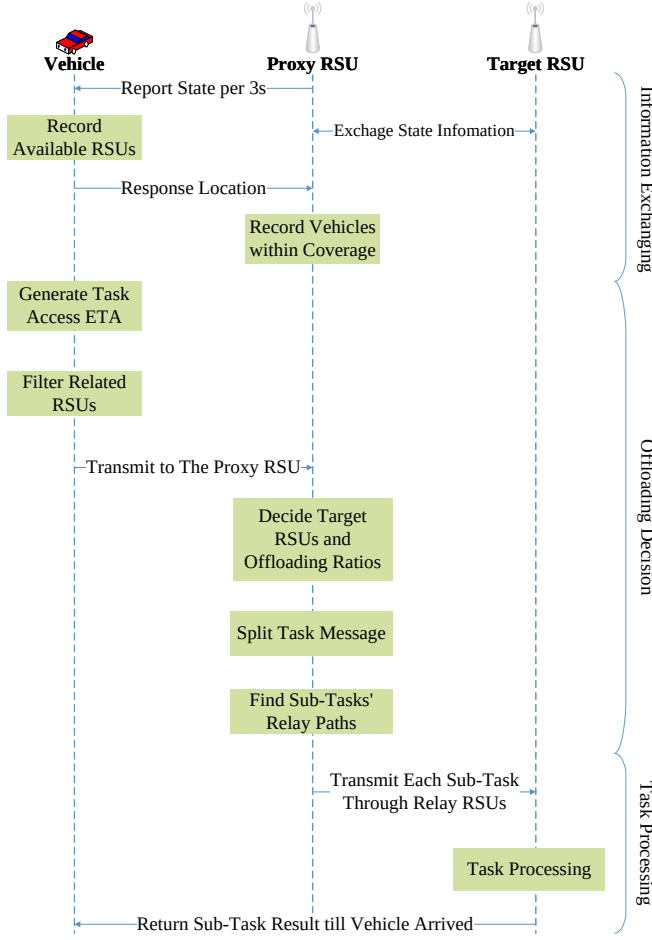


Fig. 4: The Cooperative Vehicular Task Offloading Protocol

B. Problem Model of Cooperative Vehicular Computation Offloading

In the cooperative offloading protocol, each target RSU processes and sends back a part of the task in parallel. We call the ratio of the task part processed on the target RSU as offloading ratio. Therefore, we update the offloading decision variable as $x'_{jk} \in [0, 1]^K$, where x'_{jk} represents the offloading ratio on RSU r_k . Based on this, the task QoS is

$$QoS'_j = \sum_{k=1}^{|R|} \frac{1}{L} \int_{L_{begin}}^{L_{end}} x'_{jk} \cdot T_{lk} \, dl. \quad (16)$$

As shown in equation 16, the task QoS is the weight summation of all target RSUs' sending back transmission rate. In addition, the normalization of the task QoS is

$$normQoS'_j = \frac{QoS'_j}{T_D}. \quad (17)$$

Moreover, the cooperative offloading protocol does not influence the processing delay summation of RSUs. We keep the fairness factor metric above in cooperative offloading. Then, the total performance in the cooperative offloading protocol is

$$p'_j = \alpha \cdot normQoS'_j + (1 - \alpha) \cdot normF_j. \quad (18)$$

The problem model of cooperative vehicular computation offloading is shown below.

$$\max_{x'_j} p'_j \quad (19)$$

$$\text{s.t. } x'_{jk} mem_j \leq S_k \quad (19\text{-c1})$$

$$D(n_i^{\tau_\eta}, r'_k) \leq C_m, [x'_{jk}] D(n_i^{\tau_\theta}, r_k) \leq C_m \quad (19\text{-c2})$$

$$n_i^{\tau_\eta} \leq n_i^{\tau_\theta} \leq d_j \quad (19\text{-c3})$$

$$[x'_{jk}] \cdot TD_{jk} \leq \tau_\theta - \tau_\eta \quad (19\text{-c4})$$

$$r_k \in A_j \quad (19\text{-c5})$$

$$x'_{jk} \in [0, 1], \sum x'_{jk} = 1 \quad (19\text{-c6})$$

Like the problem model of independent offloading protocol, the task t_j is proxied by RSU r'_k and processed on RSU r_k . Moreover, X'_j is the decision vector, and its element x'_{jk} represents the offloading ratio of task t_j to RSU r_k . In addition, same with 15, the integral operator in p'_j is translated to the optimization equation in 19 based on the nearest roads to RSU with vehicles' routes.

As shown in equation 19, the problem model of cooperative vehicular computation offloading can be seen as a Job-Sequence problem, an NP-Complete problem. Therefore, the cooperative vehicular computation offloading problem is NP-hard, where a greedy search strategy cannot find the optimal target RSUs within a short time. Current vehicular offloading methods are developed based on machine learning and heuristic algorithms. However, the vehicular MEC environment has a low delay requirement. Because of the long processing time, the heuristic algorithms are inappropriate for the cooperative vehicular computation offloading problem. Moreover, most tasks' available RSU lists are different, causing a highly dynamic search space. Therefore, the design of the encoding method in heuristic and machine learning algorithms is difficult, influencing the offloading performance. In conclusion, the cooperative vehicular computation offloading problem needs algorithms that can generate the target RSUs quickly.

C. Dynamic Programming Cooperative Offloading Algorithm

Based on the above problem model, we propose a cooperative offloading algorithm based on dynamic programming, named Dynamic programming maximum QoS and Fairness for Cooperative offloading (DQFC) algorithm. In DQFC, we formulate the cooperative vehicular computation offloading problem as a bin packaging problem. In detail, we divide each task into ε sub-tasks. Moreover, the task is seen as a package whose capacity is ε . The value of ε is initialized based on the processing time and task QoS requirements. The higher ε means the higher task QoS and longer processing time. In contrast, the lower ε decreases the task QoS and saves the processing time. Then, DQFC can select the combination of available RSUs to get the optimal offloading performance.

Based on the above bin packaging model, we give the definition of the state matrix in the cooperative vehicular computation offloading problem. In detail, we define the rows of the state matrix as sub-tasks' target RSU, and the columns of the state matrix are the number of unallocated sub-tasks. Then, the element of the state matrix represents the current

maximum total performance gained by the former available RSUs and current unallocated sub-tasks, shown as

$$p_sum_j^{m,n} = \max_{\beta \in m} \left(\sum_{\gamma \in n} p_sum_j^{\beta\gamma} \right), \quad (20)$$

where, $p_sum_j^{m,n}$ represents the current maximum total performance with m number of available RSUs and n number of unallocated sub-tasks. Additionally, the range of m is $\{1, 2, \dots, \varepsilon \cdot |R|\}$, and the range of n is $\{1, 2, \dots, \varepsilon\}$. Finally, the detailed definition of the state matrix is

$$State = \begin{bmatrix} p_sum_j^{1,1} & \dots & p_sum_j^{1,\varepsilon} \\ \dots & \dots & \dots \\ p_sum_j^{m,1} & \dots & p_sum_j^{m,\varepsilon} \end{bmatrix} \quad (21)$$

With the definition of the current maximum total performance and state matrix, we propose our cooperative offloading algorithm, DQFC. The detailed procedure of DQFC is shown in algorithm 2.

Algorithm 2: DQFC algorithm

input : $route_i$, the ETA results of v_i ; r'_k , proxy RSU of t_j ; A_j , set of available RSUs; $t_j = (cpu_j, mem_j, d_j)$, the task property; α , weight coefficient.

output: $Target$: the optimal target RSU; X'_j , the decision vector; L , send-back location set.

```

1  $Target \leftarrow NULL, accessState(A_j), state \leftarrow [];$ 
2 for  $r_k \in A_j$  do
3    $maxRatio[k] \leftarrow \lfloor \frac{\varepsilon \cdot S_k}{mem_j} \rfloor \cdot 1/\varepsilon;$ 
4   for  $i \in [1, \dots, maxRatio[k] \cdot \varepsilon]$  do
5      $state.append([-1] * 11);$ 
6   for  $i \in [0, \dots, \varepsilon]$  do
7      $state[0][i] \leftarrow 0;$ 
8   for  $r_k \in A_j$  do
9     for  $i \in [1, \dots, maxRatio[k] \cdot \varepsilon]$  do
10       $itemIndex \leftarrow 1 + sum(maxRatio[k-1]) + i;$ 
11      for  $j \in [0, \dots, \varepsilon]$  do
12         $state[itemIndex][j] \leftarrow state[itemIndex-1][j];$ 
13        if  $j \geq 1$  then
14           $PD_{jk}, QD_{jk}, UD_{jk}, RD_{jk} \leftarrow cpu_j/f_k \cdot$ 
15             $j/\varepsilon, \sum_{t_l \in Q_k} \frac{cpu_{t_l}}{f_k}, \frac{mem_{t_l}}{T_D}, relayPath(r'_k, r_k) \cdot$ 
16             $\frac{mem_{t_l}}{T_D};$ 
17           $TD_{jk} \leftarrow PD_{jk} + QD_{jk} + UD_{jk} + RD_{jk};$ 
18           $minDist \leftarrow C_m;$ 
19          for  $n_i^m \in route_i$  do
20            if  $D(n_i^m, r_k) < C_m$  and
21               $total\_delay_{jk} < \tau_m$  and
22               $D(n_i^m, r_k) < minDist$  then
23                 $minDist \leftarrow D(n_i^m, r_k);$ 
24                 $L.append(n_i^m.road);$ 
25                 $p'_j \leftarrow Performance(state[itemIndex][j], r_k);$ 
26                if  $p'_j > state[itemIndex][j]$  then
27                   $state[itemIndex][j] \leftarrow p'_j;$ 
28           $p'_j \leftarrow [0] * K;$ 
29          for  $r_k$  from  $r_K$  to  $r_1$  do
30            for  $i$  from  $maxRatio[k] \cdot \varepsilon$  to 1 do
31               $itemIndex \leftarrow 1 + sum(maxRatio[k-1]) + i;$ 
32              if  $state[itemIndex][w] > state[itemIndex-1][w]$  then
33                 $X'_j[k] \leftarrow X'_j[k] + 1/\varepsilon, w \leftarrow w - 1;$ 
34 return  $Target, X'_j, L$ 
```

As shown in algorithm 2, DQFC includes 3 stages: initializing, decision searching, and result generating. Firstly, in initializing stage, DQFC the available RSU list and generates the state matrix, whose elements' value is -1 (lines 1-5). In detail, DQFC accesses the limited available storage space of each available RSU and calculates the maximum number of sub-tasks each available RSU could store. Moreover, the

elements in the first row of the state matrix are set to 0 (lines 6-7). Then, in the decision searching stage, DQFC traverses the elements in the state matrix to search the offloading decision with optimal total performance (lines 8-20). For the elements with column numbers larger than 0, DQFC selects the RSU represented by the row number and calculates the total performance (lines 13-21). Then, in the total performance calculation, the summation number of allocated and unallocated sub-tasks must be equal 1. Otherwise, the total performance value will be -1 . Afterward, DQFC updates the element value of the state matrix with the total performance after the selection (lines 22-23). In the result generating stage, DQFC first traverses from the last row and column (lines 25-29). Then, DQFC selects the RSU with high element's value corresponding to the row and update the number of unallocated sub-tasks (line 29). After traversing the state matrix, DQFC updates the decision variables and returns them to the proxy RSU (lines 30).

Moreover, the time complexity of DQFC depends on the number of available RSUs and ETA results. DQFC traverses each available RSU for ε times to calculate the maximum total performance with a different number of unallocated sub-tasks. In each total performance calculation, DQFC also traverses the ETA results to find the send-back location of each target RSU. Therefore, the time complexity of DQFC is $o(\varepsilon^2 \cdot K \cdot T)$.

VII. THE IMPLEMENTATION OF THE DYNAMIC VEHICULAR OFFLOADING SIMULATION METHOD

Current vehicular offloading methods are usually simulated using Python or MATLAB with real or randomly generated datasets, lacking fine-grained vehicle trajectory and dynamic computation offloading simulation. To address this problem, we propose a fine-grained dynamic vehicular offloading simulation method based on Veins [17], a discrete vehicular network simulator. **Veins integrates the road environment and network property, where vehicles and RSUs can access all road and network information. Moreover, compared with existing vehicular network simulators, Veins simulates the transmission process of each network message and the vehicular MEC environment with more network details, which is closer to the real world. However, raw vehicle trajectory datasets are not supported by Veins. It requires the vehicle trajectories to match a specific map environment. Therefore, we apply the map-match algorithm to map the vehicle trajectories to the road environment. Moreover, Veins only implements a basic traffic event broadcasting protocol that doesn't support computation offloading simulation. To address this problem, we realize our vehicular MEC architecture in Veins application layers of vehicles and RSUs. Finally, Veins is developed in C++ and incompatible with other languages. To simplify the implementation of offloading algorithms, we also realize an outside interface to call the Python scripts. In detail, Veins operates in Java Virtual Machine and isolates with other programs, causing the incompatible problem. Therefore, to address this problem, we implemented a shared memory mechanism in our simulation method to transmit the data required by Python scripts. Moreover, the results of Python**

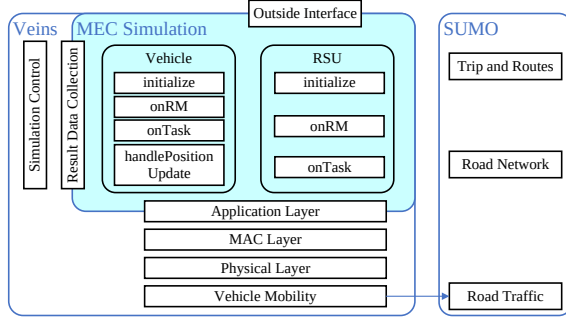


Fig. 5: Realization of Our Vehicular MEC Architecture in Veins

scripts are also transmitted through the shared memory. The detail architecture implementation are shown in Fig. 5.

As shown in Fig. 5, network connections are controlled by the Simulation Control module. In addition, the Result Data Collection module generates the log of the tasks, including the vehicles' position, target RSUs' position, and tasks' offloading ratio. Firstly, the vehicles' application layer has four functions: initialize, onRM, onTask, and handlePositionUpdate. The initialize function creates the surrounding RSU list and records the vehicle start location when vehicles are generated. In addition, for each state broadcasting message, the onRM function records the RSU state in the surrounding RSU list and responds to the vehicles' location to the RSUs. Furthermore, when vehicles receive the task results, the onTask function records the locations of source RSUs and vehicles to calculate the QoS of the task. Finally, the handlePositionUpdate function generates the tasks randomly when the vehicle changes the road, realizing the task generation and ETA service modules. In detail, when changing the road, the vehicle triggers the handlePositionUpdate function, generates the task requirements, and filters the available RSUs. After that, the task and available RSU list is transmitted to the closest RSU for further processing. Moreover, because of the limitations of Veins, the vehicles cannot access all the road information required by the ETA module. On the other hand, developing the ETA module demands accurate vehicle positions and speeds relating to drivers' privacies. The existing ETA modules are developed in secure datacenters and deployed on vehicles through encrypted module parameters, which cannot be effectively simulated in Veins. Therefore, we add the Gaussian noise to the vehicle trajectory datasets to generate the ETA results and simulate the implementation of the ETA module. In addition, we also verify the effect of ETA results' accuracy on offloading performance in the following experiments.

In RSUs' application layer, there are three functions: initialize, onRM, onTask. The initialize function here is responsible for not only state initialization, but also generating the state broadcasting messages. Then, in onRM function, the RSU handles the vehicle state message and records each vehicle's location within its coverage, realizing the information-collection module. Finally, the onTask function realizes the decision, relay, and process of tasks, realizing task scheduling,

task operation, and transmission relay modules. In detail, for the task received from the vehicles, the onTask function calls the decision algorithm to generate the target RSU locations and the send-back locations. If there are multiple target RSUs, the onTask function splits the task based on the offloading ratio and send the split tasks to the target RSUs. Then, the onTask function replays this task and transmit it to the target RSU location. When the target RSU receives the task, the onTask function appends the task to the RSU's task queue, updates the limited available storage space, and simulates the task processing. After the task processing, the onTask function releases the storage space and returns the task results to the vehicles.

VIII. EXPERIMENT

We conduct experiments on real-world datasets to answer the following key questions:

- **Q1:** How do our algorithms perform the existing vehicular offloading algorithm in processing time (see Result 1)?
- **Q2:** How does ETA Results' Accuracy affect the performance of our algorithms (see Result 2)?
- **Q3:** How does the ETA service contribute to the tasks completion ratio (see Result 3)?
- **Q4:** How does the map size of our vehicular MEC architecture affects the performance of vehicular offloading algorithms (see Result 4)?
- **Q5:** How do our algorithms perform on tasks' QoS (see Result 4)?
- **Q6:** How does the cooperation between RSUs affect the fairness factor of RSU (see Result 5)?
- **Q7:** How does the maximum task number affect the performance of offloading algorithms (see Result 6)?

A. Dataset

We test GQFI, DQFC, and other existing algorithms in public vehicle trajectory datasets of Erlangen, Germany and Shenzhen, China [18]. The vehicle trajectory dataset of Shenzhen, China includes 19542 vehicle trajectories in 15 days. In addition, each vehicle trajectory consists of its trace points' longitude, latitude, and timestamp. On the other hand, the vehicle trajectory dataset of Erlangen, Germany includes 10 most popular vehicle travel routes. We randomly generate 100 vehicles in the road map of Erlangen, Germany to verify the algorithms' performance in small areas. Furthermore, for the dataset of shenzhen, China, we match 436 vehicles to the road map to simulate the algorithms in large areas. The RSU number in Erlangen, Germany is 9, and that in Shenzhen, China is 63. Moreover, the map sizes of Erlangen, Germany and Shenzhen, China are $2.5km \times 2.5km$ and $9km \times 7km$, respectively. In conclusion, the road map and RSU number of Erlangen are relatively small, meaning a small solution space. The experiment in Erlangen could illustrate the processing time different of offloading algorithms. In contrast, Shenzhen has larger road map and more RSUs, resulting in larger solution space and discrete distributed roads. We'll focuses more on task QoS in the experiment of Shenzhen. The detailed

TABLE II: Experiment Parameters

Simulation Parameters	Values
RSU total computational capability f_k (GHz)	[3, 5]
RSU available computational capability f_k (GHz)	[1, 2]
RSU total storage capacity S_k (GB)	[16, 32]
RSU available storage capacity S_k (MB)	[70, 150]
RSU core number	4
RSU coverage C_m (m)	1000m
Task computation size per bit cpu_j (cycles/bit)	1000
Input data size of task mem_j (MB)	[5, 10]
Number of local tasks generated by a vehicle	[1, 5]
Communication transmission power P_t (mW)	250
Communication noise power P_n (mW)	0.005
Communication bandwidth W (MHz)	5
Maximum transmission rate T_D (Mbps)	60
Simulation time (s)	2000
Weight coefficient α	0.7
Sub-task number ε	10

road maps of Erlangen, Germany and Shenzhen, China are shown in Fig. 6.

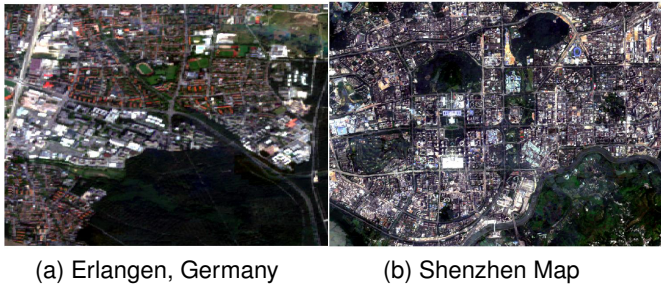


Fig. 6: The Road Maps

We input the datasets and realize our vehicular MEC architecture in Veins 5.1, including SUMO 1.8.0, OMNeT++ 5.8.2 and Python 3.8.10. Finally, we get the processing records of the RSUs and tasks to calculate the QoS and fairness factor.

In addition, we investigate the existing works on vehicular computing offloading and set the parameters of RSUs and vehicles [29] [10] [11] [42]. The detailed parameters are listed in Table II. In addition, to simulate the ETA module, we generate the ETA results from the vehicle trajectory datasets. Moreover, ETA service module can be affected by various factors, such as traffic congestion, road closures, and weather conditions. These events leads to inaccurate mobility predictions, influencing the performance of vehicular computation offloading algorithms. Therefore, we added the Gaussian noise to the ETA results to simulate the inaccurate mobility prediction. Then, the following experiment compares the effect of ETA results' accuracy on offloading algorithms. In addition, we calculate the distances between roads and RSUs, whose values are above 6 meters. Therefore, we set the value of theoretic maximum transmission rate based on the minimum distance value. Finally, the theoretic maximum transmission rate is 60Mbps.

B. Baseline Algorithms

In our experiment, we compare the following baseline algorithms with our algorithms.

Nearest [33]: Nearest is an independent offloading algorithm. For each task, Nearest selects the closest RSU as the target RSU. Finally, Nearest calculates each task's total delay and finds the service location with minimum distance between vehicles' future locations.

CooperativeGreedy [30]: CooperativeGreedy is a cooperative offloading algorithm. It calculates the total performance profit on each available RSU and sorts the available RSUs based on total performance profit. Then, CooperativeGreedy allocates the maximum offloading to each available RSU whose limited available storage capacity can support it.

Evolutionary Genetic Algorithm (EGA) [3]: EGA is a cooperative offloading algorithm. It divides each task into sub-tasks and randomly formulates the chromosomes. Then, EGA iteratively crossovers and mutates each chromosome to generate the new offloading decisions. Finally, EGA selects the best chromosome as the final offloading decision based on the chromosomes' total performance.

C. Processing Time Comparison (for Q1)

Considering the vehicular MEC environment has a low delay requirement, the solvation of independent and cooperative vehicular computation offloading problems should be as low as possible. Therefore, we compare the average processing time of offloading problem solvation in GQFI, DQFC, and baseline algorithms in the simulation environment of Erlangen, Germany. The independent offloading algorithms, Nearest and GQFI, has the fewest average processing time, which is 46 ms and 55 ms respectively. It is because the solution space of the independent offloading algorithms is much smaller than the cooperative offloading algorithms. Considering about the bias of average processing time records, the average processing time of Nearest and GQFI is close enough to be seen as equivalent. Moreover, the average processing time of CooperativeGreedy is 47 ms, which is longer than Nearest because CooperativeGreedy and Nearest only need to traverse the available RSU list. Then, because the time complexity of DQFC is higher than GQFI, DQFC has the fourth fewest average processing time, which is 155 ms. Finally, EGA has the highest average processing time, which is 453 ms. In EGA, we set the number of chromosomes as 40 and the iteration number is 100. In each iteration, EGA traverses each chromosome, including the available RSU list and vehicle route. Therefore, the time complexity of EGA in our experiment is 40×100 times of GQFI, resulting in the longest average processing time. In conclusion, the heuristic algorithms like EGA are unefficient to cooperative vehicular computation offloading. Moreover, in real IoV environments, the average processing time of offloading algorithms should be as low as can be ignored in the whole offloading procedure to satisfy the low delay requirement. The average processing time of Nearest, GQFI, CooperativeGreedy, and DQFC is below 0.2s, which is far less than the total delay. Therefore, these four offloading algorithms are more suitable to real IoV environments than EGA.

TABLE III: Completion Ratio with different ETA results' accuracy in Shenzhen, China

ETA Accuracy (%)	30	50	70	90
Completion ratio of GQFI (%)	74.0	76.1	86.4	91.7
Completion ratio of DQFC (%)	74.3	76.8	87.9	91.7

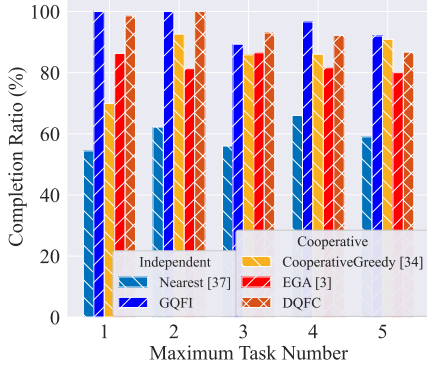


Fig. 7: Completion Ratio of Offloading Algorithms in Erlangen, Germany

D. Effect of ETA Results' Accuracy (for Q2)

When the predicted arrival time of the vehicles in ETA results is later than the real arrival time, the RSU may be failed to send the tasks' results back to the vehicle. Therefore, this section compares the completion ratio of GQFI and DQFC with different ETA results' accuracy. In addition, the maximum task number of this section is 1. The detailed completion ratio is shown in Table III.

As shown in Table III, with the ETA results' accuracy increasing, the completion ratio of GQFI and DQFC also increases. It means that the accurate arrival time prediction could effectively avoid the failure of sending back process. Moreover, we also investigate the accuracy of existing ETA modules [15] [16]. We set the ETA results' accuracy in the following experiments as 90%.

E. Completion Ratio Comparison (for Q3)

This section compares the completion ratio of GQFI, DQFC, and baseline algorithms. In detail, we see the completion ratio is the ratio between the tasks completely received by vehicles and all generated tasks. The detailed completion ratio of offloading algorithms is shown in Fig. 8.

As shown in Fig. 7, GQFI has the highest completion ratio among these five offloading algorithms. Because the send-back failure of each sub-task generated by cooperative offloading algorithms causes the failure of the whole task, DQFC and CooperativeGreedy algorithm has a lower completion ratio. In addition, EGA has a lower completion ratio than DQFC and CooperativeGreedy, which means the randomness in EGA decreases the completion ratio. Due to the lack of the ETA service, Nearest has the lowest completion ratio.

Moreover, Fig. 8 shows the offloading algorithms' completion ratios in large areas. When the maximum task number is smaller than 2, the completion ratios of offloading algorithms

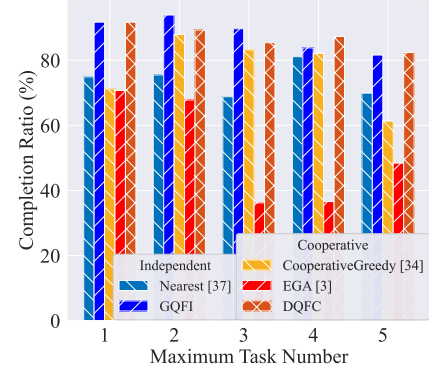


Fig. 8: Completion Ratio of Offloading Algorithms in Shenzhen, China

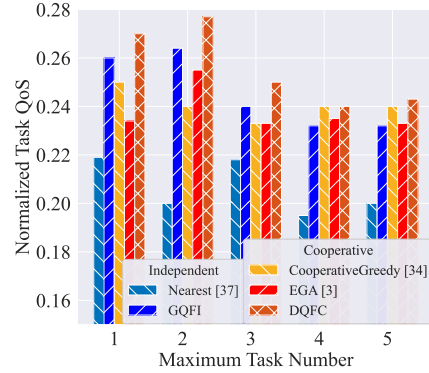


Fig. 9: Normalized QoS of Offloading Algorithms in Erlangen, Germany

are close to the results in Erlangen, Germany. However, when the maximum task number is larger than 2, EGA's completion ratio greatly decreases, below 40%. It is because EGA's number of chromosomes and iteration is too small to find the optimal offloading decision for larger solution space. Finally, DQFC's completion ratio is higher than EGA by 87.17%. In real IoV environments, our offloading algorithms could select the RSUs that can send the task results back to vehicles within their resident time, ensuring the stability of the navigation application. Therefore, the proposed algorithms could perform better than existing algorithms.

F. Task QoS Comparison (for Q4 and Q5)

To verify the optimization of task QoS, we calculate the task QoS based on equations 11 and 17. The detailed QoS performance of offloading algorithms is shown in Fig. 10.

As shown in Fig. 9, the task QoS of offloading algorithms is similar. It is because the vehicles' positions are close to all available RSUs on the road map of Erlangen, Germany. No matter what offloading decisions are generated by offloading algorithms, the transmission rate between RSUs and different vehicles' positions is close, resulting in a similar QoS task.

The results in Fig. 10 show the differences between offloading algorithms in task QoS. In detail, because of the cooperation of RSUs, DQFC has higher task QoS than GQFC and other algorithms. The sub-tasks are distributed in several

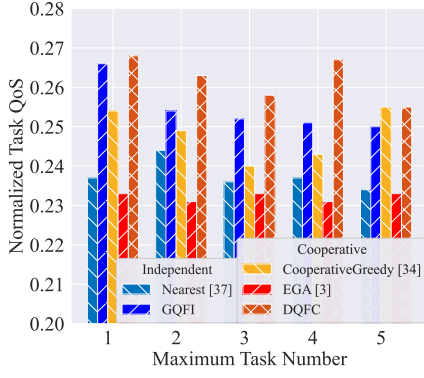


Fig. 10: Normalized QoS of Offloading Algorithms in Shenzen, China

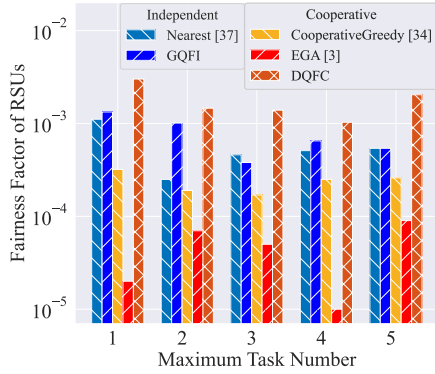


Fig. 11: Fairness Factor of RSUs in Erlangen, Germany

RSUs with similar task QoS, which saves the limited available storage space and average processing time for the other tasks and further increases the average task QoS of all tasks. However, compared with DQFC, CooperativeGreedy offloads the tasks to the RSU with the highest total performance and largest offloading ratio possible, causing a heavy workload on several frequently used RSUs and a lower task QoS than DQFC. Then, because Nearest only focuses on the task QoS, the task QoS of Nearest is just lower than CooperativeGreedy. Finally, EGA has the lowest task QoS among these offloading algorithms because of the low number of chromosomes and

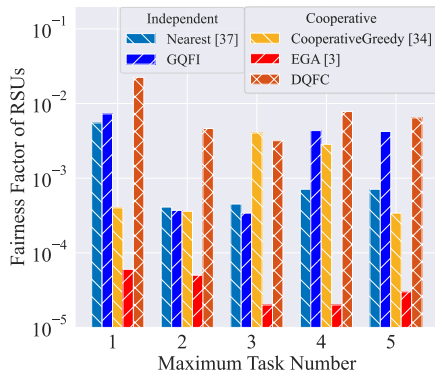


Fig. 12: Fairness Factor of RSUs in Shenzhen, China

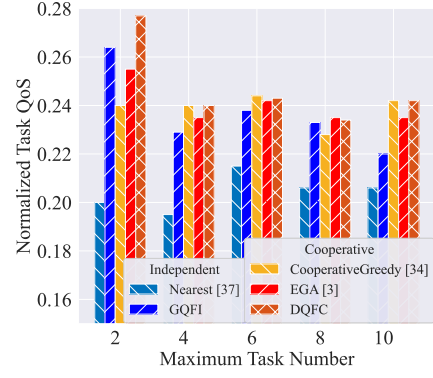


Fig. 13: Normalized QoS of Offloading Algorithms in Erlangen, Germany

iterations. In addition, the task QoS of DQFC is higher than that of EGA by 15.02%. Moreover, our offloading algorithms could optimize the transmission rate of tasks, increasing the QoS of the navigation applications in real IoV environments.

G. Fairness Factor Comparison (for Q6)

To compare the RSUs' workload distribution, we calculate the minimum fairness factors of the offloading algorithms in both two vehicle trajectory datasets. Fig. 11 and 12 show the detailed results of the fairness factor.

As shown in Fig. 11 and 12, DQFC has the highest fairness factor among these offloading algorithms, which means that DQFC effectively balances the task distribution of RSUs. Then, because GQFI considers the fairness factor in its objective function, GQFI has a fairness factor just lower than DQFC. Moreover, CooperativeGreedy has a similar fairness factor to GQFI, which means the fairness factor in total performance calculation effectively optimizes task distribution. However, the fairness factor of Nearest is also close to DQFC. In our simulation environments, the vehicles are evenly distributed in the road maps. In addition, Nearest selects the RSU with the shortest distance with the vehicles as the target RSU. Therefore, the RSUs have a similar vehicle number, meaning evenly distributed computation tasks and a high fairness factor.

Finally, the EGA has the lowest fairness factor. With the increment of task number and available RSU number, the solution space of the cooperative vehicular computation offloading problem becomes larger, decreasing the fairness factor of EGA. In addition, the fairness factor of DQFC is higher than that of Nearest by 52.82%. In conclusion, our offloading algorithm effectively balances the workload of RSUs, ensuring the stability of real IoV architectures.

H. Influence of Maximum Task Number (for Q7)

To show the influence of the maximum task number, we also calculate the QoS and fairness factors of the offloading algorithm with maximum task numbers up to 10. Fig. 13 and 14 show the detailed results of the QoS and fairness factor.

As shown in Fig. 13, the maximum task number increment decreases the QoS of the offloading algorithms. It is because the tasks use up the available resources on RSUs close to

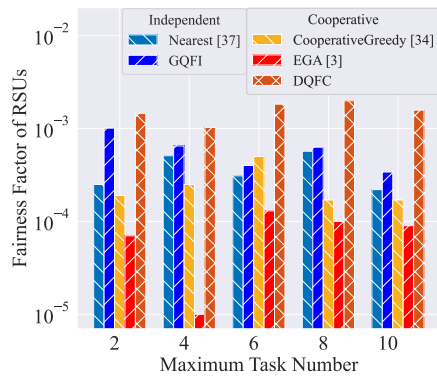


Fig. 14: Fairness Factor of RSUs in Erlangen, Germany

vehicles. Therefore, the offloading algorithms select the RSUs far from vehicles as target RSUs, resulting in a decrement in QoS. However, most vehicles reach their destinations after they generate 4 tasks. In addition, when the maximum task number exceeds 4, most RSUs drop tasks because of insufficient resources. Therefore, the QoS remains in a small range and doesn't change following the maximum task number when the maximum task number exceeds 4. Moreover, the results in Fig. 14 show that offloading algorithms' fairness factors are similar to those in Fig. 11. It means that the maximum task number has less influence on the fairness factor.

IX. CONCLUSION

This paper defines performance maximization in vehicular computation offloading problems to optimize the QoS and RSUs' response fairness. First, we propose a vehicular MEC architecture based on ETA service. Second, we propose two vehicular computation offloading algorithms, GQFI and DQFC, focusing on QoS, response fairness, and cooperation among RSUs. Experiments are conducted on the simulated IoV environment with real-world datasets. The proposed algorithms show that the task QoS and response fairness of RSUs are 15.02% and 52.82% higher than existing algorithms respectively. Moreover, results show the significant advantages in processing time and completion ratio. One promising future direction is improving resource utilization by considering offloading among vehicles.

X. ACKNOWLEDGMENTS

This paper was supported in part by the Joint Funds of the National Natural Science Foundation of China (Grant No. U22A2036); the Shenzhen Colleges and Universities Stable Support Program No. GXWD 20220817124251002. Professor Weizhe Zhang is the corresponding author (email: wz-zhang@hit.edu.cn).

REFERENCES

- [1] J. E. Siegel, D. C. Erb, and S. E. Sarma, "A Survey of the Connected Vehicle Landscape—Architectures, Enabling Technologies, Applications, and Development Areas," *IEEE Transactions on Intelligent Transportation Systems*, vol. 19, no. 8, pp. 2391–2406, Aug. 2018.
- [2] S. Zhou, P. P. Netalkar, Y. Chang, Y. Xu, and J. Chao, "The MEC-Based Architecture Design for Low-Latency and Fast Hand-Off Vehicular Networking," in *2018 IEEE 88th Vehicular Technology Conference (VTC-Fall)*, Aug. 2018, pp. 1–7.
- [3] H. Materwala, L. Ismail, R. M. Shubair, and R. Buyya, "Energy-SLA-aware genetic algorithm for edge-cloud integrated computation offloading in vehicular networks," *Future Generation Computer Systems*, vol. 135, pp. 205–222, Oct. 2022.
- [4] R. Yadav, W. Zhang, O. Kaiwartya, H. Song, and S. Yu, "Energy-latency tradeoff for dynamic computation offloading in vehicular fog computing," *IEEE Transactions on Vehicular Technology*, vol. 69, no. 12, pp. 14 198–14 211, 2020.
- [5] C. Ling, Z. Feng, L. Xu, Q. Huang, Y. Zhou, W. Zhang, and R. Yadav, "An edge server placement algorithm based on graph convolution network," *IEEE Transactions on Vehicular Technology*, pp. 1–16, 2022.
- [6] K. Nagaki, "Evolution of In-Car Navigation Systems," in *Handbook of Intelligent Vehicles*, A. Eskandarian, Ed. London: Springer, 2012, pp. 463–487.
- [7] S. Ham, J. Im, M. Kim, and K. Cho, "Construction and verification of a high-precision base map for an autonomous vehicle monitoring system," *ISPRS International Journal of Geo-Information*, vol. 8, no. 11, 2019. [Online]. Available: <https://www.mdpi.com/2220-9964/8/11/501>
- [8] W. Zhan, C. Luo, G. Min, C. Wang, Q. Zhu, and H. Duan, "Mobility-Aware Multi-User Offloading Optimization for Mobile Edge Computing," *IEEE Transactions on Vehicular Technology*, vol. 69, no. 3, pp. 3341–3356, Mar. 2020.
- [9] S. Misra and S. Bera, "Soft-VAN: Mobility-Aware Task Offloading in Software-Defined Vehicular Network," *IEEE Transactions on Vehicular Technology*, vol. 69, no. 2, pp. 2071–2078, Feb. 2020.
- [10] P. Dai, K. Hu, X. Wu, H. Xing, F. Teng, and Z. Yu, "A Probabilistic Approach for Cooperative Computation Offloading in MEC-Assisted Vehicular Networks," *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 2, pp. 899–911, 2022.
- [11] J. Shi, J. Du, J. Wang, and J. Yuan, "Deep Reinforcement Learning-Based V2V Partial Computation Offloading in Vehicular Fog Computing," in *2021 IEEE Wireless Communications and Networking Conference (WCNC)*, Mar. 2021, pp. 1–6.
- [12] Y. Li, B. Yang, H. Wu, Q. Han, C. Chen, and X. Guan, "Joint Offloading Decision and Resource Allocation for Vehicular Fog-Edge Computing Networks: A Contract-Stackelberg Approach," *IEEE Internet of Things Journal*, pp. 1–1, 2022.
- [13] A. Lakhan, M. Ahmad, M. Bilal, A. Jolfaei, and R. M. Mehmood, "Mobility Aware Blockchain Enabled Offloading and Scheduling in Vehicular Fog Cloud Computing," *IEEE Transactions on Intelligent Transportation Systems*, vol. 22, no. 7, pp. 4212–4223, Jul. 2021.
- [14] D. Dailey, "Travel-time estimation using cross-correlation techniques," *Transportation Research Part B: Methodological*, vol. 27, no. 2, pp. 97–107, 1993.
- [15] H. Hong, Y. Lin, X. Yang, Z. Li, K. Fu, Z. Wang, X. Qie, and J. Ye, "HetETA: Heterogeneous Information Network Embedding for Estimating Time of Arrival," in *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, ser. KDD '20. New York, NY, USA: Association for Computing Machinery, Aug. 2020, pp. 2444–2454.
- [16] Z. Wang, K. Fu, and J. Ye, "Learning to Estimate the Travel Time," in *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, ser. KDD '18. New York, NY, USA: Association for Computing Machinery, Jul. 2018, pp. 858–866.
- [17] C. Sommer, D. Eckhoff, A. Brummer, D. S. Buse, F. Hagenauer, S. Joerer, and M. Segata, "Veins: The Open Source Vehicular Network Simulation Framework," in *Recent Advances in Network Simulation: The OMNeT++ Environment and Its Ecosystem*, ser. EAI/Springer Innovations in Communication and Computing, A. Virdis and M. Kirsche, Eds. Cham: Springer International Publishing, 2019, pp. 215–252.
- [18] Z. Xiao, F. Li, R. Wu, H. Jiang, Y. Hu, J. Ren, C. Cai, and A. Iyengar, "Trajdata: On vehicle trajectory collection with commodity plug-and-play obu devices," *IEEE Internet of Things Journal*, vol. 7, no. 9, pp. 9066–9079, 2020.
- [19] A. Waheed, M. A. Shah, S. M. Mohsin, A. Khan, C. Maple, S. Aslam, and S. Shamshirband, "A Comprehensive Review of Computing Paradigms, Enabling Computation Offloading and Task Execution in Vehicular Networks," *IEEE Access*, vol. 10, pp. 3580–3600, 2022.
- [20] M. Wu, W. Qi, J. Park, P. Lin, L. Guo, and I. Lee, "Residual Energy Maximization for Wireless Powered Mobile Edge Computing Systems With Mixed-Offloading," *IEEE Transactions on Vehicular Technology*, vol. 71, no. 4, pp. 4523–4528, Apr. 2022.

- [21] X. Huang, S. Leng, S. Maharjan, and Y. Zhang, "Multi-Agent Deep Reinforcement Learning for Computation Offloading and Interference Coordination in Small Cell Networks," *IEEE Transactions on Vehicular Technology*, vol. 70, no. 9, pp. 9282–9293, Sep. 2021.
- [22] Z. Nan, S. Zhou, Y. Jia, and Z. Niu, "Joint Task Offloading and Resource Allocation for Vehicular Edge Computing With Result Feedback Delay," *IEEE Transactions on Wireless Communications*, pp. 1–1, 2023.
- [23] Q. Luo, C. Li, T. H. Luan, and W. Shi, "Minimizing the Delay and Cost of Computation Offloading for Vehicular Edge Computing," *IEEE Transactions on Services Computing*, vol. 15, no. 5, pp. 2897–2909, Sep. 2022.
- [24] A. Yang, J. Weng, N. Cheng, J. Ni, X. Lin, and X. Shen, "Deqos attack: Degrading quality of service in vanets and its mitigation," *IEEE Transactions on Vehicular Technology*, vol. 68, no. 5, pp. 4834–4845, May 2019.
- [25] H. Wu, J. Chen, W. Xu, N. Cheng, W. Shi, L. Wang, and X. Shen, "Delay-Minimized Edge Caching in Heterogeneous Vehicular Networks: A Matching-Based Approach," *IEEE Transactions on Wireless Communications*, vol. 19, no. 10, pp. 6409–6424, Oct. 2020.
- [26] A. Yang, J. Weng, K. Yang, C. Huang, and X. Shen, "Delegating authentication to edge: A decentralized authentication architecture for vehicular networks," *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 2, pp. 1284–1298, 2022.
- [27] H. Song, B. Gu, K. Son, and W. Choi, "Joint Optimization of Edge Computing Server Deployment and User Offloading Associations in Wireless Edge Network via a Genetic Algorithm," *IEEE Transactions on Network Science and Engineering*, pp. 1–1, 2022.
- [28] C. Chen, H. Li, H. Li, R. Fu, Y. Liu, and S. Wan, "Efficiency and Fairness Oriented Dynamic Task Offloading in Internet of Vehicles," *IEEE Transactions on Green Communications and Networking*, pp. 1–1, 2022.
- [29] C. Chen, L. Chen, L. Liu, S. He, X. Yuan, D. Lan, and Z. Chen, "Delay-Optimized V2V-Based Computation Offloading in Urban Vehicular Edge Computing and Networks," *IEEE Access*, vol. 8, pp. 18 863–18 873, 2020.
- [30] Y. Wu, J. Wu, L. Chen, J. Yan, and Y. Han, "Load Balance Guaranteed Vehicle-to-Vehicle Computation Offloading for Min-Max Fairness in VANETs," *IEEE Transactions on Intelligent Transportation Systems*, pp. 1–20, 2021.
- [31] Q. Luo, C. Li, T. H. Luan, W. Shi, and W. Wu, "Self-Learning Based Computation Offloading for Internet of Vehicles: Model and Algorithm," *IEEE Transactions on Wireless Communications*, vol. 20, no. 9, pp. 5913–5925, Sep. 2021.
- [32] Y. Xu, K. Zhu, H. Xu, and J. Ji, "Deep Reinforcement Learning for Multi-Objective Resource Allocation in Multi-Platoon Cooperative Vehicular Networks," *IEEE Transactions on Wireless Communications*, pp. 1–1, 2023.
- [33] J. Zhang, H. Guo, J. Liu, and Y. Zhang, "Task Offloading in Vehicular Edge Computing Networks: A Load-Balancing Solution," *IEEE Transactions on Vehicular Technology*, vol. 69, no. 2, pp. 2092–2104, Feb. 2020.
- [34] M. Choi, M. Shin, and J. Kim, "Dynamic video delivery using deep reinforcement learning for device-to-device underlaid cache-enabled Internet-of-vehicle networks," *Journal of Communications and Networks*, vol. 23, no. 2, pp. 117–128, Apr. 2021.
- [35] J. Zhou, C. L. P. Chen, L. Chen, and W. Zhao, "A User-Customizable Urban Traffic Information Collection Method Based on Wireless Sensor Networks," *IEEE Transactions on Intelligent Transportation Systems*, vol. 14, no. 3, pp. 1119–1128, Sep. 2013.
- [36] S. A. M. Ahmed and S. H. S. A. a. N. Fisal, "Overview of Wireless Access in Vehicular Environment (WAVE) Protocols and Standards," *Indian Journal of Science and Technology*, vol. 6, no. 7, pp. 1–8, Jun. 2013.
- [37] I. Skog and P. Handel, "In-car positioning and navigation technologies—a survey," *IEEE Transactions on Intelligent Transportation Systems*, vol. 10, no. 1, pp. 4–21, 2009.
- [38] L. Liu, M. Zhao, M. Yu, M. A. Jan, D. Lan, and A. Taherkordi, "Mobility-Aware Multi-Hop Task Offloading for Autonomous Driving in Vehicular Edge Computing and Networks," *IEEE Transactions on Intelligent Transportation Systems*, pp. 1–14, 2022.
- [39] M. Zorzi and A. Chockalingam, "Wireless Communications," in *Encyclopedia of Physical Science and Technology (Third Edition)*, R. A. Meyers, Ed. New York: Academic Press, Jan. 2003, pp. 851–874.
- [40] J. Han, M. Kamber, and J. Pei, "3 - Data Preprocessing," in *Data Mining (Third Edition)*, ser. The Morgan Kaufmann Series in Data Management Systems, J. Han, M. Kamber, and J. Pei, Eds. Boston: Morgan Kaufmann, Jan. 2012, pp. 83–124.
- [41] M. Sun, X. Xu, S. Han, H. Zheng, X. Tao, and P. Zhang, "Secure Computation Offloading for Device-Collaborative MEC Networks: A DRL-Based Approach," *IEEE Transactions on Vehicular Technology*, pp. 1–16, 2022.
- [42] J. Miranda, R. Abrishambaf, T. Gomes, P. Gonçalves, J. Cabral, A. Tavares, and J. Monteiro, "Path loss exponent analysis in Wireless Sensor Networks: Experimental evaluation," in *2013 11th IEEE International Conference on Industrial Informatics (INDIN)*, Jul. 2013, pp. 54–58.